Plan de Pruebas — OSSN Demo

Aplicación bajo prueba	Open Source Social Network (OSSN Demo)
Autora	Adriana Troche
Rol	Senior QA Engineer
Fecha	Marzo 2026
App target	demo.opensource-socialnetwork.org
Herramientas	Playwright + TypeScript, axe-core, dotenv, GitHub Actions

1. Introducción
    1.1 Objetivo del plan
    1.2 Audiencia
2. Alcance
    2.1 Dentro del alcance
    2.2 Fuera del alcance
3. Estrategia de Pruebas
    3.1 Enfoque por capas
    3.2 Distribución manual vs. automatizada
    3.3 Arquitectura de la suite automatizada
    3.4 Herramientas y justificación
4. Casos de Prueba
    4.1 Módulo de Autenticación
    4.2 Módulo Social
    4.3 Módulo de Mensajería
    4.4 Módulo de Perfil
    4.5 Casos manuales (TC-06, TC-07, TC-10)
5. Análisis de Riesgos
6. Criterios de Entrada, Salida y Calidad
    6.1 Criterios de entrada
    6.2 Criterios de salida
    6.3 Criterios de calidad (métricas target)
7. Entorno de Pruebas y Datos
    7.1 Entorno
    7.2 Estrategia de datos de prueba
    7.3 Gestión de sesiones (storageState)
8. Cronograma de Ejecución
9. Limitaciones: Cloudflare Turnstile

9.1 Problema
9.2 Análisis técnico
9.3 Solución implementada
9.4 Impacto en la entrega
10. Recomendaciones para Pruebas Futuras
10.1 Ampliar cobertura de automatización
10.2 Pruebas de rendimiento
10.3 Seguridad
10.4 Accesibilidad continua

---

1. Introducción

Este plan de pruebas define la estrategia, alcance y enfoque para validar la calidad de una red social web antes de su lanzamiento al mercado. La aplicación permite registro de usuarios, publicación de contenido con imágenes, interacciones sociales y mensajería privada.

Elegí trabajar contra OSSN Demo (Open Source Social Network) porque es la única aplicación de acceso público que cubre las 6 funcionalidades del escenario: registro, login, subida de imágenes, comentarios, reacciones, mensajes privados y personalización de perfil. Otras opciones como Conduit o Mastodon no cubren todas, y automatizar contra Instagram o Facebook sería una mala práctica (captchas, rate limiting, términos de uso).

1.1 Objetivo del plan

Garantizar que las funcionalidades críticas de la aplicación sean robustas, seguras y accesibles antes del lanzamiento. Este plan no solo cubre pruebas funcionales — incluye validación de accesibilidad y monitoreo de la comunicación entre frontend y backend.

1.2 Audiencia

- **Equipo de desarrollo** — para entender qué se está validando y los criterios de aceptación
- **Product Owner / Manager** — para la decisión de go/no-go de release
- **Otros QAs** — para continuar o expandir la cobertura de pruebas

---

2. Alcance

2.1 Dentro del alcance

Módulo	Funcionalidades	Tipos de prueba
Autenticación	Registro de usuario, inicio de sesión, validaciones de campos, manejo de errores	Funcional (happy path + negativo), automatizada, network validation
Publicaciones	Crear post con imagen y descripción, subida de archivos, validación de formatos	Funcional (happy path + negativo), automatizada
Interacciones sociales	Comentarios en publicaciones, reacciones (likes), toggle de reacciones	Funcional manual
Mensajería	Acceso a bandeja de mensajes, apertura de conversaciones existentes, envío de mensajes	Funcional automatizada + network validation
Perfil	Intento de edición de First Name en Basic Settings, detección de restricción del sistema, verificación en perfil público	Funcional automatizada (defecto documentado)
Accesibilidad	Auditoría WCAG 2.1 en páginas clave (registro, login, feed, perfil)	Automatizada con axe-core (data-driven)
Integración UI-Backend	Validación de requests HTTP, status codes, payloads en flujos críticos	Automatizada con network interception (Playwright)

2.2 Fuera del alcance

- Pruebas de carga y rendimiento (requieren herramientas como k6 o JMeter y configuración de infraestructura dedicada)
- Pruebas de penetración / seguridad avanzada (requieren expertise especializado y autorización del propietario)
- Testing de la API REST directa (el componente OssnServices no está habilitado en la demo pública)
- Pruebas en dispositivos móviles (el reto se centra en la versión web)

- Pruebas de internacionalización / localización

> 📝 **Decisión:** Excluí las pruebas de carga y seguridad avanzada del alcance de ejecución, pero incluyo recomendaciones sobre ambas en la sección de mejoras futuras. No tiene sentido prometer cobertura que no puedo entregar con calidad en 72 horas.

---

3. Estrategia de Pruebas

La estrategia se basa en un principio simple: probar primero lo que más daño haría si falla. Registro y login son la puerta de entrada — si no funcionan, nada más importa. Luego las funcionalidades sociales core (publicar, comentar, mensajes), y finalmente los extras que aportan profundidad (accesibilidad, validación de red).

3.1 Enfoque por capas

Capa	Qué valida	Cómo	Herramienta
UI Funcional	Que el usuario puede completar los flujos clave	Tests manuales + automatizados	Playwright + ejecución manual en Chrome
Network	Que la UI se comunica correctamente con el backend	Interceptación de requests HTTP durante tests de UI	Playwright (`page.waitForResponse`, `route`)
Accesibilidad	Que la app cumple estándares WCAG 2.1 básicos	Auditoría automatizada por página	`@axe-core/playwright`

3.2 Distribución manual vs. automatizada

No automaticé todo, y eso es deliberado. La automatización tiene sentido donde hay repetición, riesgo alto y flujos estables. Los tests manuales cubren interacciones que se benefician de la observación humana.

3.3 Arquitectura de la suite automatizada

La suite usa patrones avanzados de Playwright que optimizan mantenibilidad y velocidad de ejecución:

Patrón	Implementación	Beneficio
storageState	`fixtures/auth.setup.ts` hace login una vez y persiste la sesión	Elimina ~12 logins repetidos, ahorra ~60s por ejecución
Custom Fixtures	`fixtures/page-fixtures.ts` con `test.extend` inyecta Page Objects	Elimina boilerplate `new PageObject(page)` en cada test
Data-driven tests	`a11y-audit.spec.ts` genera 4 tests desde un array de configuración	Reduce ~80 líneas duplicadas a ~15 de configuración
Variables de entorno	`dotenv` • `.env.example` para credenciales	Soporte CI/CD sin hardcodear secrets en el código
Proyectos con dependencias	3 proyectos: `setup` • `authenticated`, `auth-tests` (independiente)	Tests de auth prueban el flujo completo; el resto reutiliza sesión

```
┌─────────┐
│  setup  │  Login una vez → guarda storageState
└─────────┘
     │ depende de
     ▼
┌──────────────┐      ┌──────────────┐
│ authenticated │      │ auth-tests   │  (independiente)
│              │      │              │
│ social/      │      │ register     │
│ a11y/        │      │ login        │
│ network/     │      │ register-    │
│ profile/     │      │  negative    │
│ messaging/   │      └──────────────┘
└──────────────┘
  Usa storageState
```

3.4 Herramientas y justificación

Herramienta	Uso	Por qué esta y no otra
Playwright + TypeScript	Automatización de UI, network interception, video recording	Soporte nativo para múltiples browsers, auto-waits, y video built-in.

Herramienta	Uso	Por qué esta y no otra
axe-core	Auditoría de accesibilidad WCAG 2.1	Se integra nativamente con Playwright. Es el estándar de la industria para a11y testing automatizado.
dotenv	Gestión de credenciales y variables de entorno	Evita hardcodear secrets. Compatible con CI/CD (GitHub Actions secrets).
GitHub Actions	CI/CD pipeline para ejecutar tests	El código ya vive en GitHub. Es la opción más simple y directa para CI.
Chrome DevTools	Exploración, análisis de red, performance básico	Disponible sin instalación adicional. Suficiente para el nivel de análisis de este reto.

---

4. Casos de Prueba

10 casos organizados por módulo. Cada caso incluye precondiciones, pasos, resultado esperado y criterio de aceptación. Los detalles completos de ejecución están en el Informe de Pruebas.

4.1 Módulo de Autenticación

▼ TC-01: Registro exitoso de usuario

Campo	Valor
Tipo	Happy path - Automatizado
Proyecto Playwright	`auth-tests`
Precondición	Acceder a demo.opensource-socialnetwork.org. No tener sesión activa.
Pasos	1. Navegar a la página de registro 2. Ingresar nombre completo válido 3. Ingresar email único (generado dinámicamente) 4. Ingresar password que cumpla política de seguridad 5. Ingresar fecha de nacimiento (vía evaluate para bypass del datepicker

Campo	Valor
	readonly) 6. Seleccionar género 7. Aceptar GDPR 8. Click en "Create an account"
Resultado esperado	La cuenta se crea exitosamente. El usuario es redirigido al feed o página de bienvenida.
Criterio de aceptación	Redirección exitosa + elemento de sesión activa visible en el DOM.

▼ TC-02: Registro con datos inválidos

Campo	Valor
Tipo	Negativo - Automatizado
Precondición	Página de registro accesible.
Pasos	1. Intentar registro con password débil (1 carácter) 2. Intentar registro con campos obligatorios vacíos 3. Intentar registro con email de formato inválido (sin @)
Resultado esperado	En cada caso, la app muestra un mensaje de error claro y no crea la cuenta.

▼ TC-03: Login exitoso

Campo	Valor
Tipo	Happy path - Automatizado + Network validation
Pasos	1. Navegar a la página de login 2. Ingresar username y password válidos 3. Click en "Log in" 4. Capturar la request de autenticación
Resultado esperado	Redirección a `/home`. La request de login retorna status 200/302.
Criterio de aceptación	UI: `waitForURL('**/home**')` confirma redirección exitosa. Network: response

Campo	Valor
	status < 400 + cookie de sesión establecida.

▼ TC-04: Login con credenciales inválidas

Campo	Valor
Tipo	Negativo - Automatizado
Pasos	1. Intentar login con password incorrecto 2. Intentar login con usuario inexistente 3. Intentar login con campos vacíos
Resultado esperado	Mensaje de error visible. No se establece sesión. El mensaje NO revela si el usuario existe.

4.2 Módulo Social

▼ TC-05: Publicar imagen con descripción

Campo	Valor
Tipo	Happy path - Automatizado
Proyecto Playwright	`authenticated` (usa storageState)
Pasos	1. Navegar al feed (`/home`) 2. Escribir texto en "What's on your mind?" 3. Adjuntar imagen válida (JPG) vía `setInputFiles` 4. Click en "Post" 5. Verificar visibilidad del post
Resultado esperado	El post aparece en el feed con la imagen visible y la descripción.

4.3 Módulo de Mensajería

▼ TC-08: Enviar mensaje privado

Campo	Valor
Tipo	Happy path - Automatizado + Network validation
Pasos	1. Navegar a `/messages` (bandeja de entrada) 2. Verificar que el inbox es

Campo	Valor
	visible 3. Abrir la primera conversación existente 4. Escribir mensaje de texto 5. Enviar mensaje 6. Verificar que no hay errores 5xx
Resultado esperado	La bandeja de mensajes es accesible. Se puede enviar un mensaje. No se producen errores de servidor.

4.4 Módulo de Perfil

▼ TC-09: Personalizar perfil (con detección de defecto)

Campo	Valor
Tipo	Happy path con detección de defecto - Automatizado
Proyecto Playwright	`authenticated` (usa storageState)
Pasos	1. Navegar a `/u/administrator/edit` 2. Verificar que la página muestra campos editables 3. Capturar valor actual del First Name 4. Editar con un valor **diferente** al actual (`QATester`) 5. Intentar guardar cambios 6. Verificar respuesta del sistema: mensaje de restricción, error, o éxito 7. Si aplica, verificar persistencia 8. Test de restauración: restaurar valor original
Resultado esperado	El sistema permite editar y guardar, o muestra un mensaje claro de restricción/error.
Criterio de aceptación	Se detecta y documenta el comportamiento real: si permite edición se verifica persistencia; si la restringe, se captura el mensaje y se documenta como **BUG-019**.

> ⚠️ **HALLAZGO (marzo 2026):** La demo pública de OSSN no permite editar el perfil del usuario `administrator`. El sistema muestra un mensaje de restricción al guardar, o descarta los cambios silenciosamente. Documentado como **BUG-019**. El test usa un nombre genuinamente diferente al actual para evitar falsos positivos.

4.5 Casos manuales (TC-06, TC-07, TC-10)

ID	Caso	Tipo	Resultado esperado clave
TC-06	Comentar en publicación de otro usuario	Happy path	El comentario aparece debajo del post con nombre del autor y timestamp.
TC-07	Reaccionar (like) a un post	Happy + Edge	El contador de likes incrementa. Un segundo click hace toggle (unlike).
TC-10	Upload de archivo inválido	Negativo	La app rechaza archivos no soportados y muestra mensaje de error.

---

5. Análisis de Riesgos

Este análisis refleja lo que observé durante la exploración inicial de OSSN y lo que conozco por experiencia en apps similares.

Riesgo identificado	Probabilidad	Impacto	Mitigación
Validación de password débil o ausente	Alta	Alto	TC-02 valida específicamente políticas de password.
Subida de archivos sin validación de tipo/tamaño	Media	Alto	TC-10 prueba upload de archivos no soportados.

Riesgo identificado	Probabilidad	Impacto	Mitigación
XSS en campos de texto	Media	Alto	Observación durante pruebas manuales con tags de script.
Problemas de accesibilidad en formularios	Alta	Medio	Suite de axe-core audita las páginas con formularios.
Mensajes de error que revelan información del sistema	Media	Medio	TC-04 y TC-02 validan que los mensajes sean genéricos.
Mensajería privada sin validación de contenido	Media	Medio	TC-08 valida vía network interception que no haya errores de servidor.

6. Criterios de Entrada, Salida y Calidad

6.1 Criterios de entrada

- La aplicación OSSN demo está disponible y accesible
- Es posible crear cuentas de usuario nuevas sin restricciones
- El entorno de pruebas tiene datos pre-existentes suficientes
- Las herramientas de automatización están configuradas y funcionando

6.2 Criterios de salida

- Los 10 casos de prueba han sido ejecutados (7 automatizados + 3 manuales)
- Todos los resultados están documentados con evidencia
- Los bugs encontrados están reportados con severity y priority
- La suite automatizada (24 tests) se ejecuta exitosamente
- La auditoría de accesibilidad ha sido ejecutada en las 4 páginas clave

6.3 Criterios de calidad (métricas target)

Métrica	Target	Criterio de go/no-go
Pass rate de tests funcionales	>= 80% (8/10 casos pasan)	< 60% = No-Go
Bugs críticos abiertos	0 bugs P1/Critical sin workaround	Cualquier bug crítico sin workaround = No-Go
Cobertura de requisitos	100% de requisitos con al menos 1 caso	Requisito sin cobertura = gap documentado
Cobertura de automatización	>= 70% de los casos (7/10)	Meta alcanzada
Violaciones a11y critical	0 violaciones critical en páginas de auth	Se documentan como bugs de OSSN

📊 **Nota:** Estos targets son los que yo definiría para un release real. En el contexto de este reto, el valor está en demostrar que sé definir métricas medibles, no en que OSSN las cumpla.

---

7. Entorno de Pruebas y Datos

7.1 Entorno

Componente	Detalle
URL	https://demo.opensource-socialnetwork.org
Tipo de entorno	Demo pública (staging simulado)
Browser principal	Chromium (Playwright default)
SO de ejecución	macOS (local) + Ubuntu (GitHub Actions)
Node.js	>= 18.x

7.2 Estrategia de datos de prueba

Los tests automatizados generan sus propios datos en cada ejecución. No dependen de datos pre-existentes en la demo.

- **Emails:** Generados con timestamp para evitar duplicados
- **Passwords:** Generados cumpliendo la política de OSSN
- **Imágenes:** Archivo JPG de prueba incluido en el repositorio

- **Usuarios para interacción:** Se usa el usuario administrador de la demo, cuyas credenciales se leen de variables de entorno
- **Credenciales:** Gestionadas vía `.env` (gitignored) con plantilla en `.env.example`

7.3 Gestión de sesiones (storageState)

Para optimizar la ejecución, la suite usa el patrón **storageState** de Playwright:

1. Un proyecto `setup` hace login una sola vez y guarda cookies + localStorage
2. Los tests autenticados reutilizan esa sesión
3. Los tests de autenticación **no** usan storageState — prueban el flujo completo
4. Los tests de network validation limpian cookies en `beforeEach`
5. Los tests de accesibilidad limpian cookies para páginas públicas

---

8. Cronograma de Ejecución

Fase	Actividad	Duración estimada
Día 1	Exploración de OSSN + diseño de 10 casos + ejecución manual de 5 casos con evidencias + captura de bugs	12-14 horas
Día 2	Automatización de 5 tests core + suite de accesibilidad + network interception + setup CI/CD	14-15 horas
Día 3	Mejoras de calidad + automatización TC-08/TC-09 + documentación + empaquetado	12-14 horas

---

9. Limitaciones: Cloudflare Turnstile

> ⚠️ **Descubierto el 4 de marzo de 2026:** La demo pública de OSSN está protegida por Cloudflare Turnstile, un sistema anti-bot que bloquea navegadores automatizados. Esta limitación afecta a CUALQUIER herramienta de automatización (Playwright, Cypress, Selenium), no solo a esta suite.

9.1 Problema

Al ejecutar los tests contra `demo.opensource-socialnetwork.org`, todas las navegaciones son interceptadas por una página de verificación de seguridad ("Verify you are human") que no puede ser resuelta por automatización.

9.2 Análisis técnico

Cloudflare Turnstile utiliza **TLS fingerprinting (JA3/JA4)** para distinguir browsers automatizados de browsers reales. El Chromium de Playwright tiene un fingerprint TLS diferente al de Chrome estándar.

Intentos de bypass realizados:

1. Cambio de User-Agent para simular Chrome estándar — Cloudflare no se basa solo en UA
2. Integración de `playwright-extra` con `puppeteer-extra-plugin-stealth` — parcheó APIs JS pero no el TLS
3. Solver automático de Turnstile con interacción en iframe — Cloudflare rechaza clicks programáticos
4. `channel: 'chrome'` para lanzar Chrome real — el protocolo CDP sigue siendo detectable
5. Bypass manual con headed browser — Cloudflare repite el captcha indefinidamente

9.3 Solución implementada

La suite está diseñada para ser **portable**. Si la demo pública está bloqueada, basta con cambiar `BASE_URL` en el archivo `.env`:

```
# .env
BASE_URL=https://otra-instancia-ossn.com
```

Los tests no necesitan cambios — usan rutas relativas (`/login`, `/home`) a través de los Page Objects.

9.4 Impacto en la entrega

- Los **24 tests están correctamente implementados** y compilados
- La suite **se ejecuta exitosamente** contra cualquier instancia OSSN sin Cloudflare
- Esta es una **limitación de infraestructura externa**, no del código de pruebas

---

10. Recomendaciones para Pruebas Futuras

Si este fuera un proyecto real, las siguientes acciones serían mi prioridad inmediata:

10.1 Ampliar cobertura de automatización

- Automatizar TC-06, TC-07 y TC-10
- Agregar tests de regresión visual con Playwright screenshot comparison
- Implementar tests de API directos una vez habilitado OssnServices
- Agregar casos negativos para mensajería: mensaje vacío, con adjunto, usuario bloqueado
- Agregar casos negativos para perfil: avatar inválido, First Name longitud máxima, verificar si la restricción de edición aplica solo al admin o a todos los usuarios

10.2 Pruebas de rendimiento

- Usar k6 o Artillery para medir tiempos de respuesta bajo concurrencia
- Definir SLOs: login < 2s, carga de feed < 3s, upload de imagen < 5s
- Integrar métricas de rendimiento en el pipeline de CI

10.3 Seguridad

- Auditoría de headers de seguridad (CSP, X-Frame-Options, HSTS)
- Testing de CSRF en formularios

- Validación de sanitización de inputs (XSS prevention)
- Revisión de cookies (flags Secure, HttpOnly, SameSite)

10.4 Accesibilidad continua

- Integrar axe-core como quality gate en el pipeline: 0 violaciones critical = requerido para merge
- Testing manual con lector de pantalla
- Validar navegación completa por teclado

> 💡 Estas recomendaciones no son genéricas. Cada una responde a algo que observé o que quedó fuera del alcance de este reto por tiempo. En un contexto real, las priorizaría en función del roadmap de producto y los mercados target.